Backend Development Mastery Plan

From Zero to Senior Backend Developer (24 Months)

> **Your JS Foundation**: Variables, Functions, Async/Await, Promises, Array methods, Object methods, ES6+, Modules

---

📋 How This Plan Works

Component	Purpose
Concepts	Theory you must understand
Exercises	Hands-on coding tasks (do ALL of them)
Mini Project	Apply learned concepts
Checkpoint	Self-assessment before moving on
Interview Prep	Questions you should be able to answer

🟢 PHASE 1: FOUNDATION (Months 1-6)

Month 1: Node.js Core

Week 1-2: Node.js Fundamentals

Concepts to Master

- Node.js runtime vs Browser JS
- Event Loop (single-threaded, non-blocking)
- CommonJS vs ES Modules
- File System operations
- Path handling
- Process and environment

Exercises (Complete All)

Exercise 1.1: Hello Node 🟢

```
Create a file that prints "Hello from Node.js" with current date/time
Expected output: "Hello from Node.js - 2024-01-15 10:30:45"
```

Exercise 1.2: Command Line Calculator 🟢

```
Create calculator that takes arguments from command line
Command: node calc.js add 5 3
Output: 8
Support: add, subtract, multiply, divide
Handle: Invalid inputs gracefully
```

Exercise 1.3: File Reader 🟢🟢

```
Read a text file and:
- Count total words
- Count total lines
- Find most frequent word
- Handle file not found error
```

Exercise 1.4: File Writer ⭐⭐

```
Create a logger that:
- Appends logs to a file with timestamp
- Creates new file if doesn't exist
- Rotates file when size > 1MB (create new file)
```

Exercise 1.5: Directory Scanner ⭐⭐

```
Scan a directory and list all files:
- Show file name, size, type
- Recursively scan subdirectories
- Filter by extension (e.g., only .js files)
- Output as formatted table
```

Exercise 1.6: JSON File Manager ⭐⭐⭐

```
Build a JSON data manager:
- Load JSON from file
- Add new entries
- Update existing entries
- Delete entries
- Save back to file
- Handle corrupted JSON gracefully
```

Exercise 1.7: Module System Practice ⭐⭐

```
Create a math utilities module with:
- factorial(n)
- fibonacci(n)
- isPrime(n)
- gcd(a, b)
Export using both CommonJS and ES Modules
Use in another file
```

Exercise 1.8: Environment Variables ⭐⭐

```
Create config loader that:
- Reads from .env file
- Provides defaults for missing values
- Validates required variables
- Throws clear errors for missing required vars
```

Mini Project: CLI Todo App ⭐⭐⭐

```
Requirements:
- node todo.js add "Buy groceries"
- node todo.js list
- node todo.js done 1
- node todo.js delete 1
- node todo.js search "buy"
- Store todos in todos.json
- Show completion status
- Add due dates (optional)

Bonus:
- Priority levels (high, medium, low)
- Categories/tags
- Export to CSV
```

Checkpoint 1.1

Ask yourself:

- ☐ Can I explain the event loop?
- ☐ Do I understand require() vs import?
- ☐ Can I read/write files synchronously and asynchronously?
- ☐ Can I handle errors properly?
- ☐ Can I create and use modules?

Week 3-4: HTTP & Basic Server

Concepts to Master

- HTTP protocol basics
- Request/Response cycle
- HTTP methods (GET, POST, PUT, DELETE, PATCH)
- Status codes
- Headers (Content-Type, Authorization, etc.)
- Query strings and URL parameters

Exercises (Complete All)

Exercise 1.9: Raw HTTP Server ⭐⭐

```
// Create server WITHOUT Express
// Handle 4 different routes
// Return JSON responses
// Handle 404 for unknown routes
```

Exercise 1.10: Request Logger Server ⭐⭐

```
Create server that logs every request:
- Method, URL, timestamp
- Query parameters
- Request headers
- Log to both console and file
```

Exercise 1.11: Static File Server ⭐⭐⭐

```
Serve static files from a directory:
- Handle .html, .css, .js, .json, .png, .jpg
- Set correct Content-Type headers
- Handle 404 for missing files
- Handle directory listing
```

Exercise 1.12: Simple REST API (No Express) ⭐⭐⭐

```
Build API for managing users (in-memory array):
GET /users - List all users
GET /users/:id - Get single user
POST /users - Create user (parse JSON body manually)
PUT /users/:id - Update user
DELETE /users/:id - Delete user
```

Express.js Exercises

Exercise 1.13: Express Hello World

```
Set up Express app with:
- 5 GET routes returning different data
- Proper JSON responses
- Status codes
```

Exercise 1.14: Route Parameters

```
Create routes:
GET /users/:userId
GET /users/:userId/posts/:postId
GET /products/:category/:id
Extract and validate parameters
```

Exercise 1.15: Query Parameters

```
Create search endpoint:
GET /search?q=term&category=books&page=1&limit=10
Parse all query params
Return filtered results from sample data
Handle missing/invalid params
```

Exercise 1.16: Request Body Handling

```
Create endpoints that accept JSON body:
POST /calculate - { operation, numbers: [] }
POST /transform - { text, transformations: ['upper', 'reverse'] }
Validate body structure
Return proper errors for invalid input
```

Mini Project: Books API

```
Build complete CRUD API for books:
- POST /books - Add book { title, author, year, isbn }
- GET /books - List all (support ?author=X, ?year=2020)
- GET /books/:id - Get single book
- PUT /books/:id - Update book
- DELETE /books/:id - Delete book
- GET /books/search?q=term - Search in title/author

Store in memory (array), proper error handling.
Return proper status codes (200, 201, 400, 404).
```

Checkpoint 1.2

- ☐ Can I create HTTP server without Express?
- ☐ Do I understand all HTTP methods?
- ☐ Can I parse URL parameters and query strings?
- ☐ Can I handle JSON request body?
- ☐ Can I set proper status codes?

Month 2: Express.js Deep Dive

Week 1-2: Middleware & Error Handling

Concepts to Master

- Middleware pattern
- Request lifecycle
- Built-in middleware
- Custom middleware
- Error handling middleware
- Middleware order matters

Exercises (Complete All)

Exercise 2.1: Logger Middleware ⭐⭐

```
// Create middleware that logs:
// [2024-01-15 10:30:45] GET /users - 200 - 45ms
// Method, URL, status code, response time
// Log to both console and file
```

Exercise 2.2: Authentication Middleware ⭐⭐⭐

```
Create middleware that:
- Checks for API key in header (x-api-key)
- Validates API key against allowed list
- Blocks request if invalid
- Adds user info to req object if valid
- Returns proper error response if missing/invalid
```

Exercise 2.3: Request Validator Middleware ⭐⭐⭐

```
Create configurable validator:
const validateBody = (schema) => (req, res, next) => { ... }

// Usage:
app.post('/users', validateBody({
  name: { required: true, type: 'string', minLength: 2 },
  email: { required: true, type: 'email' },
  age: { required: false, type: 'number', min: 0 }
}), createUser);
```

Exercise 2.4: Rate Limiter Middleware ⭐⭐⭐

```
Build rate limiter from scratch:
- Max 100 requests per IP per minute
- Return 429 Too Many Requests when exceeded
- Include Retry-After header
- Store counts in memory (Map)
- Clean up old entries periodically
```

Exercise 2.5: Response Formatter Middleware ⭐⭐

```
Create middleware that standardizes all responses:
{
  success: true/false,
  data: {...},
  error: null or { message, code },
  timestamp: "2024-01-15T10:30:45Z"
}
```

Exercise 2.6: Error Handler Middleware ⭐⭐⭐

```
Create global error handler:
- Catches all errors
- Logs error details
- Returns user-friendly message
- Different response in dev vs prod
- Handles different error types:
  - ValidationError -> 400
  - NotFoundError -> 404
  - UnauthorizedError -> 401
  - Default -> 500
```

Exercise 2.7: Middleware Chain ⭐⭐⭐

```
Create chain of middleware:
1. Request ID generator (adds unique ID to each request)
2. Logger (uses request ID)
3. Auth checker
4. Rate limiter
5. Body validator
6. Route handler
7. Response formatter
8. Error handler

Test full chain works correctly
```

Mini Project: Blog API with Middleware ⭐⭐⭐⭐

```
Features:
- CRUD for posts
- CRUD for comments on posts
- All middleware from exercises
- Proper error handling
- Request logging
- Rate limiting
- Response formatting

Endpoints:
POST /posts
GET /posts
GET /posts/:id
PUT /posts/:id
DELETE /posts/:id
POST /posts/:id/comments
GET /posts/:id/comments
DELETE /posts/:id/comments/:commentId
```

Checkpoint 2.1

- ☐ Can I create custom middleware?
- ☐ Do I understand middleware execution order?
- ☐ Can I handle errors globally?
- ☐ Can I implement rate limiting?
- ☐ Can I validate request data?

Week 3-4: File Uploads & Static Files

Exercises

Exercise 2.8: Static File Server ⭐⭐

```
Configure Express to serve:
- Static files from /public
- Custom 404 page for missing files
- Cache headers for images
- Gzip compression for text files
```

Exercise 2.9: Single File Upload ⭐⭐

```
Using Multer:
- Upload single image
- Validate file type (jpg, png only)
- Limit file size to 5MB
- Generate unique filename
- Return file URL
```

Exercise 2.10: Multiple File Upload ⭐⭐⭐

```
- Upload up to 5 images at once
- Validate each file
- Process files (resize to max 800px width)
- Return all URLs
- Handle partial failures
```

Exercise 2.11: File Upload with Progress ⭐⭐⭐

```
Create upload endpoint that:
- Accepts large files (up to 100MB)
- Stores in chunks
- Reports progress via Server-Sent Events
- Allows resume on failure
```

Exercise 2.12: Avatar Upload System ⭐⭐⭐

```
- Upload profile picture
- Crop to square (center crop)
- Resize to 200x200
- Convert to webp
- Delete old avatar
- Return new URL
```

Mini Project: Image Gallery API ⭐⭐⭐⭐

```
Build image gallery backend:
- Upload images (multiple at once)
- List all images with pagination
- Get image by ID
- Delete image
- Image transformations via URL params:
  GET /images/123?width=300&height=200&format=webp
- Thumbnail generation
- Store original + generate thumbnails
- Handle all image formats (jpg, png, gif, webp)
```

Checkpoint 2.2

- ☐ Can I handle file uploads with Multer?
- ☐ Can I validate file types and sizes?
- ☐ Can I serve static files with proper caching?
- ☐ Can I resize/transform images?

Month 3: MongoDB & Mongoose

Week 1-2: MongoDB Basics

Concepts to Master

- Document database vs Relational
- Collections and Documents
- BSON data types
- Query operators
- Indexes
- Aggregation basics

MongoDB Exercises (Use MongoDB Shell or Compass)

Exercise 3.1: Basic CRUD

```
Create database "practice"
Create collection "users"
- Insert 10 users with: name, email, age, city, createdAt
- Find all users
- Find users where age > 25
- Update user's city
- Delete user by id
```

Exercise 3.2: Query Operators

```
Using users collection:
- Find users aged 20-30
- Find users in cities: "NYC", "LA", "Chicago"
- Find users where name starts with "A"
- Find users created in last 7 days
- Find users with email containing "gmail"
```

Exercise 3.3: Update Operators

```
- Increment all users' age by 1
- Add "verified" field to all users
- Push new item to user's "hobbies" array
- Remove item from array
- Rename field across all documents
```

Exercise 3.4: Aggregation Pipeline

```
- Count users per city
- Find average age per city
- Find cities with more than 5 users
- Group users by age range (20-30, 30-40, etc.)
- Find top 3 cities by user count
```

Exercise 3.5: Indexing

```
- Create index on email (unique)
- Create compound index on city + age
- Check query performance with explain()
- Compare query time with/without index
```

Exercise 3.6: 50 Query Challenge

```
Create "ecommerce" database with:
- products (name, price, category, stock, rating)
- orders (userId, products[], total, status, date)
- users (name, email, address, orders[])

Write queries for:
1-10: Basic CRUD on all collections
11-20: Filtering with multiple conditions
21-30: Update operations
31-40: Aggregation queries
41-50: Complex queries with $lookup (joins)
```

Week 3-4: Mongoose ORM

Concepts to Master

- Schema definition
- Data types and validators
- Virtuals and methods
- Statics
- Pre/post hooks
- Population (refs)

Exercises

Exercise 3.7: Schema Design ⭐⭐

```
// Create schemas for:
// User: name, email (unique), password, role, profile { avatar, bio }
// Post: title, content, author (ref User), tags[], likes (number)
// Comment: text, author (ref User), post (ref Post)

// Add validators:
// - email format
// - password min length 8
// - role enum ['user', 'admin']
```

Exercise 3.8: Schema Methods ⭐⭐⭐

```
// Add instance methods:
userSchema.methods.comparePassword = async function(password) { ... }
userSchema.methods.generateAuthToken = function() { ... }

// Add static methods:
userSchema.statics.findByEmail = function(email) { ... }
userSchema.statics.findAdmins = function() { ... }
```

Exercise 3.9: Virtuals ⭐⭐

```
// Add virtual for full name
userSchema.virtual('fullName').get(function() {
  return `${this.firstName} ${this.lastName}`;
});

// Add virtual for post count
userSchema.virtual('postCount', { ... });
```

Exercise 3.10: Hooks (Middleware) ⭐⭐⭐

```
// Pre-save: hash password
userSchema.pre('save', async function(next) { ... });

// Pre-remove: delete all user's posts
userSchema.pre('remove', async function(next) { ... });

// Post-save: send welcome email (mock)
userSchema.post('save', function(doc) { ... });
```

Exercise 3.11: Population ⭐⭐⭐

```
// Get post with author details
Post.findById(id).populate('author', 'name email')

// Get post with comments and comment authors
Post.findById(id)
  .populate('author')
  .populate({
    path: 'comments',
    populate: { path: 'author' },
  })
```

Exercise 3.12: Custom Queries ⭐⭐⭐

```
// Create query helpers:
postSchema.query.byAuthor = function(userId) { ... };
postSchema.query.published = function() { ... };
postSchema.query.recent = function(days) { ... };

// Usage:
Post.find().byAuthor(userId).published().recent(7);
```

Mini Project: Blog Platform API ⭐⭐⭐⭐

```
Models: User, Post, Comment, Category

Features:
- User registration (password hashed)
- Create/edit/delete posts (only author)
- Posts have categories (many-to-many)
- Comments on posts
- Like posts
- Get posts by category
- Get user's posts
- Search posts
- Pagination

Bonus:
- Draft/published status
- Featured posts
- View count
- Tags
```

Checkpoint 3

- ☐ Can I design MongoDB schemas?
- ☐ Can I write complex queries?
- ☐ Can I use aggregation pipeline?
- ☐ Can I implement relations with populate?
- ☐ Can I use hooks and methods?

Month 4: Authentication & Security

Week 1-2: JWT Authentication

Concepts to Master

- Password hashing (bcrypt)
- JWT structure (header.payload.signature)
- Token generation and verification
- Token expiration
- Refresh tokens
- Secure token storage

Exercises

Exercise 4.1: Password Hashing 🔐

```
// Implement:
const hashPassword = async (password) => { ... }
const verifyPassword = async (password, hash) => { ... }

// Test with different passwords
// Verify same password produces different hashes (salt)
```

Exercise 4.2: JWT Basics 🔐

```
// Create token
const generateToken = (payload, expiresIn) => { ... }

// Verify token
const verifyToken = (token) => { ... }

// Decode token (without verification)
const decodeToken = (token) => { ... }

// Test with different payloads and expiration times
```

Exercise 4.3: Signup Flow 🔐🔐

```
POST /auth/signup
{
  name: "John",
  email: "john@example.com",
  password: "securepass123"
}

Steps:
1. Validate input
2. Check if email exists
3. Hash password
4. Create user
5. Generate token
6. Return user + token
```

Exercise 4.4: Login Flow 🔐🔐

```
POST /auth/login
{
  email: "john@example.com",
  password: "securepass123"
}

Steps:
1. Find user by email
2. Compare password
3. Generate token
4. Return user + token
Handle: Invalid email, wrong password
```

Exercise 4.5: Auth Middleware ⭐⭐⭐

```
// Create middleware that:
// 1. Extracts token from Authorization header
// 2. Verifies token
// 3. Attaches user to req
// 4. Handles expired/invalid tokens

const protect = async (req, res, next) => { ... };

// Usage:
app.get('/profile', protect, getProfile);
```

Exercise 4.6: Role-Based Access ⭐⭐⭐

```
// Create middleware for role checking
const authorize = (...roles) => (req, res, next) => { ... };

// Usage:
app.delete('/users/:id', protect, authorize('admin'), deleteUser);
app.get('/admin/dashboard', protect, authorize('admin', 'moderator'), getDashboard);
```

Exercise 4.7: Password Reset Flow ⭐⭐⭐⭐

```
1. POST /auth/forgot-password { email }
   - Generate reset token
   - Save hashed token + expiry in DB
   - Send email with reset link (mock)

2. POST /auth/reset-password/:token { password }
   - Verify token is valid and not expired
   - Hash new password
   - Update user
   - Clear reset token
```

Mini Project: Complete Auth System ⭐⭐⭐⭐

```
Endpoints:
POST /auth/signup
POST /auth/login
POST /auth/logout
POST /auth/forgot-password
POST /auth/reset-password/:token
GET /auth/me (protected)
PUT /auth/update-password (protected)
PUT /auth/update-profile (protected)

Features:
- Password hashing
- JWT tokens
- Protected routes
- Password reset with email
- Update password
- Update profile

Security:
- Rate limit login attempts
- Password strength validation
- Email verification (bonus)
```

Week 3-4: Advanced Authentication

Exercises

Exercise 4.8: Refresh Token System ⭐⭐⭐⭐

```
Implement:
1. Access token (15 minutes)
2. Refresh token (7 days)
3. POST /auth/refresh to get new access token
4. Store refresh tokens in database
5. Invalidate refresh tokens on logout
6. Token rotation (new refresh token on each use)
```

Exercise 4.9: Device Management ⭐⭐⭐⭐

```
Features:
- Track all logged-in devices
- Store: device name, IP, last active
- GET /auth/devices - list all devices
- DELETE /auth/devices/:id - logout specific device
- DELETE /auth/devices - logout all devices
```

Exercise 4.10: OAuth Preparation ⭐⭐⭐

```
// Understand OAuth flow:
// 1. Redirect user to provider
// 2. User authenticates
// 3. Provider redirects back with code
// 4. Exchange code for tokens
// 5. Get user info
// 6. Create/link local account

// Implement mock OAuth flow to understand the process
```

Exercise 4.11: Two-Factor Authentication ⭐⭐⭐⭐

```
Implement 2FA with TOTP:
1. Enable 2FA - generate secret, show QR code
2. Verify 2FA - user enters code from app
3. Login with 2FA - require code after password
4. Disable 2FA
5. Backup codes for recovery
```

Exercise 4.12: Session vs Token ⭐⭐⭐

```
Implement same auth with sessions:
- express-session
- Session storage (Redis)
- Compare with JWT approach
- Understand pros/cons
```

Mini Project: OAuth Social Login ⭐⭐⭐⭐⭐

```
Implement login with:
- Google OAuth
- GitHub OAuth

Features:
- Link social account to existing account
- Handle "email already exists"
- Get profile picture from OAuth
- Proper error handling
```

Checkpoint 4

- ☐ Can I hash and verify passwords?
- ☐ Can I generate and verify JWTs?
- ☐ Can I implement protected routes?
- ☐ Can I handle refresh tokens?
- ☐ Can I implement password reset?
- ☐ Do I understand OAuth flow?

Month 5: Advanced API Development

Week 1-2: Validation & Error Handling

Concepts to Master

- Schema validation (Zod/Joi)
- Custom error classes
- Error handling patterns
- Logging strategies
- API response standards

Exercises

Exercise 5.1: Zod Schemas ⭐⭐

```
```

```
// Create schemas for:
const userSchema = z.object({
  name: z.string().min(2).max(50),
  email: z.string().email(),
  password: z.string().min(8).regex(/[A-Z]/).regex(/[0-9]/),
  age: z.number().int().min(18).max(120).optional(),
  role: z.enum(['user', 'admin']).default('user'),
  profile: z
    .object({
      bio: z.string().max(500).optional(),
      website: z.string().url().optional(),
    })
    .optional(),
})

// Test with valid and invalid data
```

Exercise 5.2: Validation Middleware ⬜⬜⬜

```
// Create reusable validation middleware
const validate = schema => (req, res, next) => {
  // Validate req.body, req.query, req.params
  // Return formatted errors
}

// Usage:
app.post('/users', validate(userSchema), createUser)
```

Exercise 5.3: Custom Error Classes ⬜⬜⬜

```
// Create error classes:
class AppError extends Error { ... }
class ValidationError extends AppError { ... }
class NotFoundError extends AppError { ... }
class UnauthorizedError extends AppError { ... }
class ForbiddenError extends AppError { ... }
class ConflictError extends AppError { ... }

// Each should have: message, statusCode, code
```

Exercise 5.4: Global Error Handler ⬜⬜⬜⬜

```
// Handle:
// - Custom AppError
// - Mongoose ValidationError
// - Mongoose CastError (invalid ObjectId)
// - Mongoose DuplicateKey
// - JWT errors
// - Multer errors
// - Unknown errors

// Different response for dev vs prod
```

Exercise 5.5: Logging System ⬜⬜⬜

```
// Using Winston:
// - Log to console (dev)
// - Log to files (prod)
// - Different log levels
// - Log rotation
// - Request logging
// - Error logging with stack trace
```

Exercise 5.6: API Response Format ⬜⬜

```
// Standardize all responses:

// Success:
{
  success: true,
  data: { ... },
  meta: { page, limit, total } // for lists
}

// Error:
{
  success: false,
  error: {
    message: "User-friendly message",
    code: "VALIDATION_ERROR",
    details: [{ field: "email", message: "Invalid email" }]
  }
}
```

Mini Project: Validated Blog API ⭐⭐⭐⭐

```
Take existing Blog API and add:
- Zod validation for all inputs
- Custom error classes
- Global error handler
- Logging system
- Standardized responses

Test all error scenarios:
- Invalid input
- Not found
- Unauthorized
- Duplicate entry
- Server error
```

Week 3-4: Pagination, Filtering & Search

Exercises

Exercise 5.7: Basic Pagination ⭐⭐

```
GET /posts?page=2&limit=10

Return:
{
  data: [...],
  pagination: {
    page: 2,
    limit: 10,
    total: 100,
    pages: 10,
    hasNext: true,
    hasPrev: true
  }
}
```

Exercise 5.8: Cursor Pagination ⭐⭐⭐

```
GET /posts?cursor=abc123&limit=10

// Better for real-time data
// Use last item's ID as cursor
// Implement forward and backward navigation
```

Exercise 5.9: Filtering ⭐⭐⭐

```
GET /products?
  category=electronics&
  minPrice=100&
  maxPrice=500&
  inStock=true&
  brand=apple,samsung

// Build dynamic filter object
// Handle multiple values
// Validate filter fields
```

Exercise 5.10: Sorting ⭐⭐

```
GET /products?sort=price&order=desc
GET /products?sort=-price,name  // Multiple fields

// Support ascending/descending
// Multiple sort fields
// Validate sort fields
```

Exercise 5.11: Search ⭐⭐⭐

```
GET /products?search=iphone

// Search in: name, description
// Case insensitive
// Highlight matching text
// Relevance scoring (basic)
```

Exercise 5.12: Combined Query Builder ⭐⭐⭐⭐

```
// Create reusable query builder:
class APIFeatures {
  filter() { ... }
  sort() { ... }
  paginate() { ... }
  search() { ... }
  select() { ... }  // Field selection
}

// Usage:
const features = new APIFeatures(Product.find(), req.query)
  .filter()
  .search()
  .sort()
  .paginate();

const products = await features.query;
```

Mini Project: E-commerce Product API ⭐⭐⭐⭐

```
Products with:
- name, description, price, category, brand
- stock, rating, numReviews
- images[], variants[]

Endpoints:
GET /products (with full query features)
GET /products/:id
GET /products/categories (list all categories)
GET /products/stats (count per category, avg price, etc.)

Query Examples:
/products?category=electronics&minPrice=100&sort=-rating&page=1
/products?search=iphone&brand=apple&inStock=true
```

Checkpoint 5

- ☐ Can I validate input with Zod?
- ☐ Can I handle errors globally?
- ☐ Can I implement pagination?
- ☐ Can I filter and sort results?
- ☐ Can I implement search?

Month 6: File Storage & Email

Week 1-2: Cloud Storage

Exercises

Exercise 6.1: Cloudinary Setup ⭐⭐

```
// Setup Cloudinary
// Upload image from local file
// Get public URL
// Delete image
// List all images
```

Exercise 6.2: Upload from API ⭐⭐⭐

```
// POST /upload
// 1. Receive file via multer
// 2. Upload to Cloudinary
// 3. Save URL to database
// 4. Delete local file
// 5. Return URL
```

Exercise 6.3: Image Transformations ⭐⭐⭐

```
// Generate on-the-fly:
// - Thumbnails (150x150)
// - Medium (500x500)
// - Large (original)
// - WebP format
// - Different quality levels
```

Exercise 6.4: Signed Uploads ⭐⭐⭐

```
// Client uploads directly to Cloudinary:
// 1. Backend generates signed URL
// 2. Frontend uploads to signed URL
// 3. Backend receives webhook notification
// 4. Save URL to database
```

Exercise 6.5: Multiple Storage Support ⭐⭐⭐⭐

```
// Create storage abstraction:
class StorageService {
  upload(file) { ... }
  delete(url) { ... }
  getUrl(key) { ... }
}

class CloudinaryStorage extends StorageService { ... }
class S3Storage extends StorageService { ... }
class LocalStorage extends StorageService { ... }

// Switch storage via env var
```

Week 3-4: Email System

Exercises

Exercise 6.6: Nodemailer Setup ⭐⭐

```
// Setup transporter
// Send simple text email
// Send HTML email
// Handle errors
```

Exercise 6.7: Email Templates ⭐⭐⭐

```
// Create templates for:
// - Welcome email
// - Password reset
// - Order confirmation
// - Newsletter

// Use Handlebars or EJS
// Pass dynamic data
```

Exercise 6.8: Email Service Abstraction ⭐⭐⭐

```
class EmailService {
  async send(to, template, data) { ... }
  async sendWelcome(user) { ... }
  async sendPasswordReset(user, token) { ... }
  async sendOrderConfirmation(order) { ... }
}
```

Exercise 6.9: Email Queue ⭐⭐⭐⭐

```
// Don't send email in request:
// 1. Add email job to queue
// 2. Worker processes queue
// 3. Retry on failure
// 4. Log results

// Use simple queue (array + setInterval) for now
// Upgrade to Bull in Month 11
```

Exercise 6.10: Email Provider Fallback

```
// If primary provider fails:
// 1. Try secondary provider
// 2. Log failure
// 3. Alert admin
```

Mini Project: Social Media API

```
Features:
- User registration with email verification
- Profile with avatar (Cloudinary)
- Posts with images (multiple)
- Password reset via email
- Welcome email

Models:
- User (with avatar URL from Cloudinary)
- Post (with images array from Cloudinary)

Email Templates:
- Welcome
- Email verification
- Password reset
```

Checkpoint 6 (Phase 1 Complete!)

- ☐ Can I upload files to cloud storage?
- ☐ Can I transform images?
- ☐ Can I send emails with templates?
- ☐ Can I handle email failures?

Phase 1 Capstone Project: Full-Featured Blog Platform

Build complete blog platform with everything learned:

```
Models:
- User (auth, profile, avatar)
- Post (title, content, images, category, tags)
- Comment (text, author, post)
- Category

Features:
- Full authentication (JWT, refresh tokens)
- Email verification
- Password reset
- Profile with avatar upload
- CRUD posts with images
- Comments on posts
- Categories and tags
- Full-text search
- Pagination, filtering, sorting
- Proper validation
- Error handling
- Logging

API Endpoints: 25+ endpoints

Deploy to Railway or Render
```

🗄️ PHASE 2: PROFESSIONAL (Months 7-12)

Month 7: PostgreSQL & Prisma

Week 1-2: SQL Fundamentals

Exercises

Exercise 7.1: SQL Basics 🟢

```
-- Create tables, insert, select, update, delete
-- 20 queries on single table
```

Exercise 7.2: JOINs 🟡🟡

```
-- INNER, LEFT, RIGHT, FULL JOIN
-- 15 queries with joins
```

Exercise 7.3: Aggregations 🟡🟡

```
-- COUNT, SUM, AVG, MIN, MAX
-- GROUP BY, HAVING
-- 15 aggregate queries
```

Exercise 7.4: Subqueries 🟡🟡🟡

```
-- Subqueries in SELECT, WHERE, FROM
-- 10 complex queries
```

Exercise 7.5: 50 SQL Query Challenge 🟡🟡🟡

```
E-commerce database:
- users, products, orders, order_items, categories

50 queries covering:
- CRUD operations
- JOINs
- Aggregations
- Subqueries
- Window functions
```

Week 3-4: Prisma ORM

Exercises

Exercise 7.6: Prisma Setup ⭐⭐

```
// Define schema:
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String?
  posts     Post[]
  createdAt DateTime @default(now())
}

model Post {
  id        Int      @id @default(autoincrement())
  title     String
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
}
```

Exercise 7.7: Relations ⭐⭐⭐

```
// Implement:
// - One-to-many (User -> Posts)
// - Many-to-many (Post <-> Tag)
// - One-to-one (User -> Profile)
// - Self-relation (User -> followers/following)
```

Exercise 7.8: Prisma Queries ⭐⭐⭐

```
// Practice all query types:
// create, findUnique, findMany, update, delete
// include, select
// where with operators
// orderBy, take, skip
// Aggregations
```

Exercise 7.9: Transactions ⭐⭐⭐

```
// Implement:
// - Transfer money between accounts
// - Create order with multiple items
// - Handle failure (rollback)
```

Mini Project: E-commerce API (PostgreSQL) ⭐⭐⭐⭐⭐

```
Models:
- User, Product, Category
- Order, OrderItem
- Cart, CartItem
- Review

Features:
- Full CRUD
- Relations
- Transactions for orders
- Complex queries
```

Month 8: Real-Time Features

Week 1-2: Socket.io

Exercises

Exercise 8.1: Socket.io Setup ⭐⭐

```
// Server: Accept connections, handle disconnect
// Client: Connect, emit, listen
```

Exercise 8.2: Chat Room ⭐⭐⭐

```
// Join room, leave room
// Broadcast to room
// Private messages
// User list per room
```

Exercise 8.3: Typing Indicator ⭐⭐⭐

```
// Show "User is typing..."
// Debounce typing events
// Clear when stopped typing
```

Exercise 8.4: Online Status ⭐⭐⭐

```
// Track online users
// Broadcast status changes
// Handle disconnection
```

Mini Project: Real-Time Chat App ⭐⭐⭐⭐⭐

```
Features:
- User authentication
- Multiple chat rooms
- Private messaging
- Typing indicators
- Online status
- Message history
- Read receipts
- File sharing
```

Week 3-4: SSE & Notifications

Exercises

Exercise 8.5: Server-Sent Events ⭐⭐⭐

```
// Implement SSE endpoint
// Send periodic updates
// Handle client reconnection
```

Exercise 8.6: Notification System ⭐⭐⭐⭐

```
// Create notification service:
// - Store notifications in DB
// - Send real-time via Socket.io
// - Mark as read
// - Get unread count
// - Notification preferences
```

Month 9: Security & Testing

Week 1-2: API Security

Exercises

Exercise 9.1: Security Headers ⭐⭐

```
// Add all security headers with helmet
// Understand each header's purpose
```

Exercise 9.2: Rate Limiting ⭐⭐⭐

```
// Different limits for:
// - Login endpoint
// - Password reset
// - General API
// - Per user limits
```

Exercise 9.3: Input Sanitization ⭐⭐⭐

```
// Prevent XSS
// Prevent NoSQL injection
// Prevent SQL injection
// Sanitize file uploads
```

Exercise 9.4: Security Audit ⭐⭐⭐⭐

```
// Check your existing API:
// - OWASP Top 10
// - Authentication vulnerabilities
// - Authorization bypass
// - Data exposure
// - Rate limiting
// Document and fix issues
```

Week 3-4: Testing

Exercises

Exercise 9.5: Unit Testing ⭐⭐

```
// Test utility functions
// Test validators
// Test services (mock dependencies)
```

Exercise 9.6: Integration Testing ⬜⬜⬜

```
// Test API endpoints
// Use test database
// Test auth flows
// Test error cases
```

Exercise 9.7: E2E Testing ⬜⬜⬜⬜

```
// Test complete user flows:
// - Signup -> Login -> Create Post -> Comment
// - Add to cart -> Checkout -> Payment
```

Exercise 9.8: 80% Coverage Challenge ⬜⬜⬜⬜

```
// Write tests to achieve 80% code coverage
// for complete Blog API
```

Month 10: Caching & Performance

Week 1-2: Redis Caching

Exercises

Exercise 10.1: Redis Basics ⬜⬜

```
// SET, GET, DEL
// Expiration (TTL)
// Lists, Sets, Hashes
```

Exercise 10.2: API Response Caching ⬜⬜⬜

```
// Cache middleware:
// - Check cache before DB
// - Store response in cache
// - Set appropriate TTL
// - Cache invalidation on update
```

Exercise 10.3: Session Storage ⬜⬜⬜

```
// Store sessions in Redis
// Compare with JWT
```

Exercise 10.4: Caching Strategies ⬜⬜⬜⬜

```
// Implement:
// - Cache aside
// - Write through
// - Write behind
// Understand when to use each
```

Week 3-4: Database Optimization

Exercises

Exercise 10.5: Query Analysis ⬜⬜⬜

```
// Use explain() / EXPLAIN
// Identify slow queries
// Add appropriate indexes
// Measure improvement
```

Exercise 10.6: N+1 Problem ⬛⬛⬛

```
// Identify N+1 queries
// Fix with populate/include
// Use DataLoader (for advanced)
```

Exercise 10.7: Performance Benchmark ⬛⬛⬛⬛

```
// Benchmark your API:
// - Response times
// - Throughput
// - Identify bottlenecks
// - Optimize and re-benchmark
```

Month 11: Background Jobs

Week 1-2: Job Queues (Bull)

Exercises

Exercise 11.1: Bull Setup ⬛⬛

```
// Create queue
// Add job
// Process job
// Handle completion/failure
```

Exercise 11.2: Email Queue ⬛⬛⬛

```
// Queue email jobs
// Process with worker
// Retry failed emails
// Track status
```

Exercise 11.3: Job Scheduling ⬛⬛⬛

```
// Schedule jobs:
// - Daily report at midnight
// - Weekly cleanup
// - Monthly invoice generation
```

Exercise 11.4: Job Dashboard ⬛⬛⬛

```
// Set up Bull Board
// Monitor queues
// Retry failed jobs
// Clear completed jobs
```

Week 3-4: Scheduled Tasks

Exercises

Exercise 11.5: Cron Jobs ⬛⬛

```
// Using node-cron:
// - Daily database backup
// - Hourly metrics collection
// - Weekly report email
```

Exercise 11.6: Task Orchestration ⬛⬛⬛⬛

```
// Complex task workflows:
// End of day processing:
// 1. Generate reports
// 2. Send emails
// 3. Archive data
// 4. Update statistics
```

Month 12: Payments & Deployment

Week 1-2: Stripe Integration

Exercises

Exercise 12.1: Stripe Setup ⬛⬛

```
// Create payment intent
// Handle successful payment
// Handle failed payment
```

Exercise 12.2: Webhooks ⬛⬛⬛

```
// Handle webhook events:
// - payment_intent.succeeded
// - payment_intent.payment_failed
// - Verify webhook signature
```

Exercise 12.3: Subscription System ⬛⬛⬛⬛

```
// Create subscription plans
// Subscribe user to plan
// Handle subscription events
// Cancel subscription
```

Mini Project: E-commerce with Payments ⬛⬛⬛⬛⬛

```
Complete e-commerce:
- Product catalog
- Shopping cart
- Checkout with Stripe
- Order management
- Payment history
- Refunds
```

Week 3-4: Production Deployment

Exercises

Exercise 12.4: Environment Config ⬛⬛

```
// Proper .env management
// Different configs for dev/staging/prod
// Secrets handling
```

Exercise 12.5: Railway/Render Deployment ⭐⭐

```
// Deploy Node.js app
// Connect to production DB
// Set environment variables
// Custom domain
// HTTPS
```

Exercise 12.6: Monitoring Setup ⭐⭐⭐

```
// Set up:
// - Error tracking (Sentry)
// - Logging (Winston + cloud)
// - Health checks
// - Uptime monitoring
```

Exercise 12.7: CI/CD Pipeline ⭐⭐⭐⭐

```
// GitHub Actions:
// - Run tests
// - Lint code
// - Deploy if tests pass
// - Notify on failure
```

🏆 Phase 2 Capstone: Full E-commerce Platform

```
Features:
- User auth (JWT + refresh tokens)
- Product catalog (PostgreSQL + Prisma)
- Shopping cart
- Order management
- Payment (Stripe)
- Real-time order updates (Socket.io)
- Email notifications
- Background job processing
- Redis caching
- Full test coverage
- CI/CD pipeline
- Production deployed
```

🔴 PHASE 3: SENIOR LEVEL (Months 13-24)

Months 13-14: Microservices

Topics & Exercises

Exercise 13.1: Split Monolith ⭐⭐⭐⭐

```
Split e-commerce into services:
- User Service
- Product Service
- Order Service
- Payment Service
- Notification Service

Each service:
- Own database
- REST API
- Independent deployment
```

Exercise 13.2: API Gateway ⭐⭐⭐⭐

```
// Create gateway that:
// - Routes requests to services
// - Handles authentication
// - Rate limiting
// - Request logging
```

Exercise 13.3: Service Communication ⭐⭐⭐⭐

```
// Implement:
// - REST calls between services
// - Service discovery (basic)
// - Circuit breaker
// - Retry logic
```

Months 15-16: Message Queues & Events

Topics & Exercises

Exercise 15.1: RabbitMQ Basics ⭐⭐⭐

```
// Publish/Subscribe
// Work queues
// Topic exchanges
```

Exercise 15.2: Event-Driven Architecture ⭐⭐⭐⭐

```
// Order created ->
//   - Reduce inventory
//   - Send confirmation email
//   - Update analytics
// All via events, not direct calls
```

Months 17-18: GraphQL

Topics & Exercises

Exercise 17.1: Apollo Server Setup ⭐⭐⭐

```
// Schema definition
// Resolvers
// Queries and Mutations
```

Exercise 17.2: DataLoader ⭐⭐⭐⭐

```
// Solve N+1 problem
// Batch data fetching
```

Exercise 17.3: GraphQL API ⬜⬜⬜⬜⬜

```
// Complete Blog API in GraphQL
// Authentication
// Subscriptions for real-time
```

Months 19-20: Docker

Topics & Exercises

Exercise 19.1: Dockerfile ⬜⬜⬜

```
# Multi-stage build
# Optimize image size
# Security best practices
```

Exercise 19.2: Docker Compose ⬜⬜⬜⬜

```
# Full stack:
# - Node.js app
# - PostgreSQL
# - Redis
# - Nginx
```

Exercise 19.3: Containerized Microservices ⬜⬜⬜⬜⬜

```
// Containerize all microservices
// Docker network
// Orchestration basics
```

Months 21-22: System Design

Topics & Exercises

Exercise 21.1: Design URL Shortener ⬜⬜⬜

```
Requirements:
- Shorten URLs
- Redirect to original
- Analytics

Consider:
- Scale to 1B URLs
- High read volume
```

Exercise 21.2: Design Notification System ⬜⬜⬜⬜

```
Requirements:
- Multi-channel (push, email, SMS)
- Millions of users
- Preferences

Consider:
- Queue architecture
- Rate limiting
- Persistence
```

Exercise 21.3: Design Rate Limiter ⬛⬛⬛⬛

```
Requirements:
- Distributed
- Different limit types
- Low latency

Algorithms:
- Token bucket
- Sliding window
- Fixed window
```

Exercise 21.4: Design File Storage System ⬛⬛⬛⬛⬛

```
Requirements:
- Upload/download files
- Share files
- Sync across devices

Consider:
- Chunked uploads
- Deduplication
- CDN integration
```

Months 23-24: Specialization

Option 1: AWS Services

```
- S3, EC2, Lambda
- RDS, DynamoDB
- API Gateway
- CloudWatch
```

Option 2: Kubernetes

```
- Deployments, Services
- Ingress
- ConfigMaps, Secrets
- Scaling
```

Option 3: Serverless

```
- AWS Lambda / Vercel Functions
- Event-driven
- Cost optimization
```

📊 Progress Tracking

Phase 1 Checklist (Months 1-6)

- ☐ Node.js fundamentals
- ☐ Express.js middleware
- ☐ MongoDB + Mongoose
- ☐ JWT Authentication
- ☐ Input validation
- ☐ File uploads
- ☐ Email sending
- ☐ Capstone project deployed

Phase 2 Checklist (Months 7-12)

- ☐ PostgreSQL + Prisma
- ☐ Socket.io real-time
- ☐ API security
- ☐ Testing (80% coverage)
- ☐ Redis caching
- ☐ Background jobs
- ☐ Stripe payments
- ☐ Production deployment
- ☐ CI/CD pipeline

Phase 3 Checklist (Months 13-24)

- ☐ Microservices architecture
- ☐ Message queues
- ☐ GraphQL
- ☐ Docker
- ☐ System design
- ☐ Specialization

🎯 You Are "Pro" When You Can:

✅ Build any API from scratch in < 3 hours ✅ Debug production issues confidently ✅ Design database schema without hesitation ✅ Implement auth in 30 minutes ✅ Optimize slow queries instantly ✅ Handle 10K+ concurrent users ✅ Explain architectural decisions clearly ✅ Code review others confidently ✅ Design scalable systems on whiteboard ✅ Mentor junior developers

---

Remember: Progress > Perfection. Build, break, learn, repeat!